

CaloEye — Bản trình bày ngắn phần Hạ tầng/Ops (trả lời hội đồng)

Dùng khi đứng trước hội đồng — ngắn, dễ nói, dễ nhớ. Bản đầy đủ/sâu để tự ôn xem DEPLOY-DEFENSE.md. Cả nhóm chỉ có 30 phút cho toàn bộ app, phần ops chỉ nên nói chủ động 3–5 phút, còn lại dành cho hỏi–đáp.

A. Thông số kỹ thuật — tra cứu nhanh

Mục	Giá trị
Gói VPS	Cheap 6
CPU	4 vCPU
RAM	8 GB
Ổ đĩa	55 GB SSD NVMe
Băng thông	200 Mbps, data transfer không giới hạn
Backup hạ tầng	Auto backup hàng ngày (tính năng của nhà cung cấp VPS, mức snapshot toàn máy)
OS	Ubuntu 22.04.1 LTS (kernel 5.15, x86_64)
Mức dùng thực tế	Disk ~18% / 55GB, RAM ~13% / 8GB (còn nhiều head-room)
Domain	caloeye.xyz (DNS quản lý tại Mắt Bão)
SSL	Let's Encrypt (certbot), gia hạn tự động qua cron
Registry image	Docker Hub

Thư mục app trên VPS	/var/www/app (clone repo, không chỉ chứa image)
----------------------	---

B. 5 container & cấp phát RAM

Container	Image	Vai trò (1 dòng)	RAM giới hạn
postgres	postgres:16-alpine	Database	256 MB
backend	build riêng (Laravel API)	PHP-FPM chạy API, migrate lúc start	256 MB
scheduler	cùng image backend	Cron job nội bộ (nhắc uống nước, streak, thông báo...)	128 MB
queue	cùng image backend	Xử lý job nền (mail, push, gọi AI)	128 MB
nginx	nginx:alpine	Reverse proxy, SSL, serve file tĩnh	64 MB
Tổng cấp phát			~832 MB / 8 GB

Vì sao cấp phát vậy: giới hạn cứng theo `mem_limit` trong Docker Compose để 1 container lỗi (memory leak) không nuốt hết RAM của cả máy; tổng cấp phát chỉ ~10% RAM có sẵn, còn lại là head-room cho OS, PM2, và burst traffic — không cần scale tách máy ở quy mô đồ án. postgres/backend cấp nhiều nhất vì giữ state (buffer cache DB) và xử lý request chính; nginx nhẹ nhất vì chỉ proxy/serve tĩnh.

Sửa ở đâu nếu cần đổi số RAM: giá trị thật nằm ở `docker-compose.prod.yml`, khóa `mem_limit` khai báo riêng cho từng service (postgres, backend, scheduler, queue, nginx) — tìm bằng Ctrl+F "mem_limit". Bảng trên chỉ là bản chụp lại để nói cho nhanh, sửa số thật phải sửa file này rồi deploy lại (không sửa được ở file trình bày này).

C. Kịch bản nói (~3–5 phút)

Viết dạng văn nói để dễ thuộc — nói tự nhiên theo ý, không cần học thuộc lòng từng chữ.

1. Vấn đề (30s): "Phần em phụ trách là hạ tầng và vận hành cho CaloEye. App là Laravel API + Vue 3 SPA, gộp chung 1 app. Khác với project mẫu nhóm từng làm bằng Node.js, Laravel đọc cấu hình .env lúc chạy chứ không đóng gói sẵn vào image, nên em phải tự thiết kế lại kiến trúc container cho phù hợp."

2. Kiến trúc (60–90s): "Em chia thành 5 container chạy độc lập qua Docker Compose: postgres cho database, backend chạy API Laravel, scheduler chạy các tác vụ định kỳ như nhắc uống nước hay tính streak, queue xử lý việc nền như gửi mail hay gọi AI, và nginx làm reverse proxy kiêm SSL termination, trả file tĩnh cho phần Vue. Ảnh người dùng upload và dữ liệu database được lưu ở Docker volume riêng, tách khỏi vòng đời container — nên mỗi lần deploy, container cũ bị xóa tạo lại nhưng dữ liệu không mất."

3. CI/CD (60–90s): "Quy trình tự động hoàn toàn: khi em push code lên nhánh main, GitHub Actions build image — build cả frontend lẫn backend gộp vào 1 image để đơn giản hóa việc quản lý version — rồi push lên Docker Hub. Sau đó pipeline tự SSH vào VPS bằng SSH key, pull code cấu hình mới, pull image mới, rồi khởi động lại toàn bộ stack. Toàn bộ không cần em thao tác tay trên server."

4. Cơ chế restart khớp CI/CD (45–60s): "Điểm em muốn nhấn mạnh là cách tắt–bật khi deploy: vì có tới 5 container cần dừng và khởi động lại đúng thứ tự, em dùng PM2 chạy ngay trên máy chủ (không phải trong container) để quản lý cả cụm. Khi deploy, PM2 gửi tín hiệu dừng, script bắt tín hiệu đó và chạy docker compose down để tắt sạch cả 5 container theo đúng thứ tự, sau đó PM2 tự khởi động lại, docker compose up chạy với image mới vừa pull về. Cách này đảm bảo không có tình trạng container cũ và mới cùng tranh port hay cùng ghi vào 1 volume."

5. Chốt (15–20s): "Toàn bộ hạ tầng này đang chạy thật ở caloeye.xyz, đã qua nhiều lần deploy tự động mà không mất dữ liệu."

D. Kỹ thuật / khái niệm nổi bật đã áp dụng

(nói ngắn khi được hỏi "có dùng kỹ thuật/thuật toán gì không" — mỗi ý 1 câu, không đi sâu)

- **Multi-stage Docker build** — build frontend (Node) và backend (PHP) ở 2 giai đoạn riêng trong cùng 1 Dockerfile, giai đoạn cuối chỉ giữ lại kết quả cần thiết, không mang theo toolchain build.
- **Tách bake-time vs runtime config** — biến không nhạy cảm (URL API) được ghi cứng lúc build, còn secret (API key, mật khẩu DB) chỉ đọc từ .env trên server lúc container khởi động, không bao giờ đi qua CI/CD hay image.
- **Named volume cho persistence** — dữ liệu (DB, ảnh upload) sống trong Docker volume độc lập với vòng đời container, đây là cách chuẩn để container "stateless" mà dữ liệu vẫn bền vững.

- **Healthcheck làm điều kiện khởi động (dependency ordering)** — container backend chỉ được coi là "khỏe" sau khi migrate DB xong, các service phụ thuộc (queue, scheduler, nginx) chỉ start sau khi backend khỏe, tránh chạy app khi schema DB chưa sẵn sàng.
- **Chia sẻ file tĩnh giữa 2 container qua symlink + volume** — vì nginx và backend là 2 container tách biệt filesystem, ảnh upload được "bắc cầu" qua volume dùng chung, nginx chỉ đọc (read-only).
- **Reverse proxy + Let's Encrypt (giao thức ACME)** — nginx terminate SSL, chứng chỉ tự gia hạn qua cron, không downtime khi reload.
- **Rollback theo tag commit-sha** — mỗi lần build, image được gắn thêm tag theo mã commit (ngoài tag latest), cho phép quay lại đúng 1 phiên bản cụ thể khi cần, giống ý tưởng immutable release versioning.

```

• # 1. Xem lịch sử commit để chọn commit muốn rollback về
• git log --oneline -10
•
• # 2. Pull image ứng với đúng commit đó (tag = commit-sha, đã tự động push kèm :latest lúc build)
• docker pull <dockerhub_username>/caloeye-backend:<COMMIT_SHA>
•
• # 3. Trỏ .env sang tag đó thay vì :latest
• cd /var/www/app
• sed -i 's/^DOCKER_IMAGE=.*\/DOCKER_IMAGE=<dockerhub_username>\/caloeye-backend:<COMMIT_SHA>/' .env
•
• # 4. Restart để áp dụng (PM2 tự down/up lại cả 5 container với image vừa trỏ)
• pm2 restart caloeye
•
• # --- Sau khi hết sự cố, quay lại bản mới nhất ---
• sed -i 's/^DOCKER_IMAGE=.*\/DOCKER_IMAGE=<dockerhub_username>\/caloeye-backend:latest/' .env

```

```
pm2 restart caloeye
```

Nói khi trình bày: cơ chế này đã chuẩn bị sẵn sàng (image có tag theo commit-sha, .env có thể trỏ tùy ý) — nếu chưa từng chạy thật trên production thì nên nói rõ "cơ chế đã sẵn sàng, chưa có dịp phải rollback thật" thay vì khẳng định đã test, để trả lời trung thực nếu hội đồng hỏi sâu.

- **Graceful shutdown qua SIGTERM/trap** — script deploy bắt tín hiệu dừng để chủ động tắt sạch trước khi container mới lên, thay vì kill cứng.

E. Lệnh nào set up ở file nào

(trả lời khi bị hỏi "cấu hình deploy nằm ở đâu, sửa gì ở đâu")

File	Việc gì nằm trong đó
.github/workflows/deploy.yml (job deploy)	Kích hoạt chuỗi deploy trên VPS qua SSH, chạy tuần tự: git pull origin main → docker pull <image>:latest → pm2 restart caloeys → docker image prune -f. Bản thân docker compose down/up không nằm ở đây.
ecosystem.config.cjs	Khai báo app PM2 tên caloeys, chạy scripts/start.sh bằng bash, kill_timeout: 30000 — cho containers tối đa 30s để tắt sạch trước khi PM2 force-kill.
scripts/start.sh	Nơi thật sự chứa docker compose up (chạy foreground — đây là tiến trình PM2 đang theo dõi) và docker compose down (trong trap cleanup SIGTERM SIGINT, chạy khi PM2 gửi tín hiệu dừng).

Luồng nối tiếp khi pm2 restart caloeys chạy: PM2 gửi SIGTERM cho tiến trình docker compose up đang chạy trong start.sh → trap bắt được → chạy docker compose down (tắt sạch cả 5 container) → PM2 thấy tiến trình cũ đã thoát → tự khởi động lại start.sh → docker compose up chạy lại, lần này dùng image mới vừa docker pull ở bước trước trong deploy.yml.

Muốn sửa gì thì sửa đúng chỗ: đổi bước SSH/thứ tự lệnh CI/CD → sửa deploy.yml; đổi cách PM2 quản lý (timeout, tên process) → sửa ecosystem.config.cjs; đổi cách containers tắt/bật (ví dụ thêm log, đổi cách cleanup) → sửa scripts/start.sh.

F. Câu hỏi hội đồng dự đoán + trả lời ngắn

Q: Cấu hình máy chủ thế nào, tại sao chọn gói này?

VPS 4 vCPU / 8GB RAM / 55GB SSD NVMe. Toàn bộ 5 container chỉ cấp phát khoảng 830MB RAM, nên gói này dư sức cho quy mô đồ án, còn nhiều head-room để tăng tải hoặc thêm service sau này.

Q: RAM/CPU có bị thiếu khi nhiều người dùng cùng lúc không?

Hiện tại mức dùng thực tế chỉ ~13% RAM lúc bình thường, còn nhiều dư địa. Nếu tải tăng thật thì hướng mở rộng là tăng RAM/CPU cho VPS (scale-up) trước, vì kiến trúc hiện tại chưa cần scale nhiều máy.

Q: Cơ chế "auto backup hàng ngày" hoạt động thế nào?

Đây là tính năng snapshot toàn máy do nhà cung cấp VPS cung cấp sẵn, chưa phải cơ chế backup dữ liệu do em tự cấu hình ở tầng ứng dụng (như dump DB định kỳ ra nơi khác) — đây là điểm em nhận diện là hướng cải thiện nếu có thêm thời gian.

Q: Vì sao chia 5 container riêng thay vì 1 container chạy hết?

Để cô lập trách nhiệm và giới hạn tài nguyên riêng cho từng phần (DB, API, job nền, cron, proxy) — 1 phần bị lỗi/rớt không kéo sập toàn bộ, và có thể theo dõi log/restart riêng từng phần.

Q: File ảnh người dùng upload lưu ở đâu, có mất khi deploy không?

Lưu trong Docker volume riêng, tách khỏi vòng đời container. Mỗi lần deploy, container bị xóa tạo lại nhưng volume vẫn còn — ảnh không mất.

Q: Secret như API key, mật khẩu DB có bị lộ qua GitHub/Docker Hub không?

Không. Các secret chỉ nằm trong file .env trên server, đọc lúc container chạy — không commit, không build vào image, không đi qua pipeline CI/CD.

Q: Vì sao dùng PM2 chạy trên server thay vì để Docker tự khởi động lại container?

Vì cần điều phối tắt/bật đúng thứ tự cho cả cụm 5 container cùng lúc mỗi lần deploy (đổi sang image mới) — không chỉ đơn giản là restart 1 container.

Q: Nếu deploy lỗi (ví dụ migration lỗi) thì xử lý sao?

Nếu migrate lỗi, container backend không "khỏe", các service phụ thuộc sẽ không khởi động theo — tránh chạy app sai schema. Em xem log để sửa, hoặc rollback về image của commit trước đó bằng tag đã lưu sẵn.

Q: Điểm yếu / hướng cải thiện nếu có thêm thời gian?

Chưa có backup dữ liệu đưa ra ngoài VPS (chỉ có snapshot của nhà cung cấp), chưa có staging riêng (deploy thẳng production), và chưa có health-check/alerting chủ động — hiện chỉ xem log thủ công khi có sự cố.

Tài liệu dùng khi trình bày/trả lời hội đồng — bản rút gọn từ DEPLOY-DEFENSE.md. Nguồn số liệu container/CI-CD: DEPLOY-DEFENSE.md, docker-compose.prod.yml, .github/workflows/deploy.yml, ecosystem.config.cjs, scripts/start.sh. Nguồn thông số VPS: thông tin gói dịch vụ "Cheap 6".